

WEEK 5 DELIVERABLE

Forecasting Models and Evaluation Summary

Energy Consumption Analytics Project for GreenPower Utilities
IIT Jodhpur • Group 5
Submitted: July 3, 2026

Team Members

Name	Roll No.	Email
Harshit Nirmal Jain	G25AI1021	g25ai1021@iitj.ac.in
K R Devika	G25AI1022	g25ai1022@iitj.ac.in
Kartik Dadhich	G25AI1023	g25ai1023@iitj.ac.in
Kirtiman Sarangi	G25AI1024	g25ai1024@iitj.ac.in
Kolipara Teja	G25AI1025	g25ai1025@iitj.ac.in

Table of Contents

1. Introduction and Scope
2. Modelling Approach and Data Split
3. Baseline: Seasonal-Naive Forecast
4. SARIMA Model
5. LSTM Model
6. Model Comparison and Evaluation
 - 6.1 Error Metrics
 - 6.2 Forecast vs Actual
7. Error Analysis
8. Anomaly Detection
 - 8.1 Method
 - 8.2 Results
9. Generation Forecasting
10. Reproducibility and Code
11. Limitations
12. Summary and Next Steps

1. Introduction and Scope

This report documents the fifth week of the GreenPower Utilities capstone and opens Phase 3. Weeks 1 and 2 acquired and cleaned the data, Week 3 loaded it into TimescaleDB, and Week 4 turned it into analysis-ready feature tables. This week we use those feature tables to build the analytics the project was ultimately for: short-term demand forecasting and anomaly detection. The deliverable for the week, as set out in the brief, is the modelling code and an evaluation summary.

The forecasting task is defined precisely so that the evaluation is meaningful. We predict the household's hourly active power one hour ahead across a held-out test period, using only information available before the predicted hour: the lag, rolling, calendar, and degree-hour features materialised in Week 4. We build three models of increasing sophistication — a seasonal-naive baseline, a SARIMA statistical model, and an LSTM neural network — and compare them on the same hold-out with the same metrics, so that the value added by each step is visible rather than assumed. In parallel we build an anomaly detector to flag the sudden drops and spikes that indicate outages or sensor faults, which the brief calls out explicitly.

Everything in this report reads from the Week 4 feature tables and adds no new raw data. The models are trained on the consumption track, which is the longer and richer of the two series; a short section at the end applies the same framework to wind generation. All code is committed to the project repository alongside the earlier phases.

2. Modelling Approach and Data Split

Time-series models must never be evaluated on data that leaked from the future, so the split is chronological rather than random. We train on the first part of the series and hold out the final 60 days (1,440 hourly observations) as a test set the models never see during fitting. The 33,168 earlier hours form the training set, and the split falls on 2010-09-27.

Split	Period	Hours	Purpose
Training	2006-12-16 → 2010-09-27	33,168	Fit model parameters
Test (hold-out)	2010-09-27 → 2010-11-26	1,440	Evaluate one-hour-ahead forecasts

Every model is scored on the identical test window with three complementary metrics: mean absolute error (MAE) in kilowatts, root-mean-square error (RMSE) in kilowatts, which penalises large misses more heavily, and mean absolute percentage error (MAPE), which expresses the error in relative terms. The features feeding the models are exactly those built in Week 4 — the 24-hour and 168-hour lags, the 24-hour rolling mean and standard deviation, the cyclical hour and day encodings, and the heating degree-hours — so the modelling reuses the feature layer rather than recomputing anything.

3. Baseline: Seasonal-Naive Forecast

A forecasting project needs a baseline that any real model must beat, otherwise a good-looking error number means nothing. Because household demand repeats weekly, the natural baseline is the seasonal-naive forecast: the prediction for a given hour is simply the value observed exactly one week (168 hours) earlier. This is the lag_168h feature from Week 4 used directly as a prediction.

```
-- seasonal-naive: last week, same hour
```

```
SELECT time, active_power_kwh AS actual, lag_168h AS forecast
FROM consumption_features_hourly
WHERE time > DATE '2010-09-27';
```

On the hold-out the seasonal-naive baseline scores an MAE of 0.26 kW, an RMSE of 0.339 kW, and a MAPE of 25.7%. That is already a respectable forecast — weekly structure is strong — and it sets the bar the statistical and neural models must clear to justify their added complexity.

4. SARIMA Model

The first genuine model is SARIMA (Seasonal AutoRegressive Integrated Moving Average), the standard statistical approach for a series with trend and seasonality. It extends the baseline by learning how recent hours and recent errors combine, on top of the daily and weekly seasonal cycles. We fit it with a daily seasonal period and selected the orders by minimising AIC on the training set, arriving at a parsimonious specification.

```
import pmdarima as pm
model = pm.auto_arima(train_y,
                      seasonal=True, m=24,          # daily seasonality
                      d=0, D=1,                    # one seasonal difference
                      max_p=3, max_q=3,            # search bounds
                      information_criterion='aic',
                      stepwise=True, suppress_warnings=True)
# selected: SARIMA(2,0,1)(1,1,1)[24]
```

SARIMA improves clearly on the baseline, cutting RMSE to 0.228 kW and MAPE to 19.4%. It captures the smooth daily rhythm well; where it struggles is the sharpest evening peaks, which it tends to smooth, and that residual is exactly what the neural model is meant to address.

5. LSTM Model

The third model is an LSTM (Long Short-Term Memory) recurrent neural network, which can learn non-linear interactions between demand and its drivers that a linear statistical model cannot. It reads a rolling window of the previous 168 hours together with the Week 4 features — the cyclical time encodings and the heating degree-hours — and predicts the next hour. Inputs are min-max scaled, and training uses early stopping on a validation slice to avoid over-fitting.

```
model = Sequential([
    LSTM(64, input_shape=(168, n_features), return_sequences=False),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1)                                     # one-hour-ahead active power
])
model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, validation_split=0.15,
          epochs=30, batch_size=64, callbacks=[EarlyStopping(patience=5)])
```

Training converges smoothly, with the validation loss tracking the training loss and flattening by roughly the twentieth epoch, which indicates a good fit rather than memorisation. Figure 1 shows the loss curves.

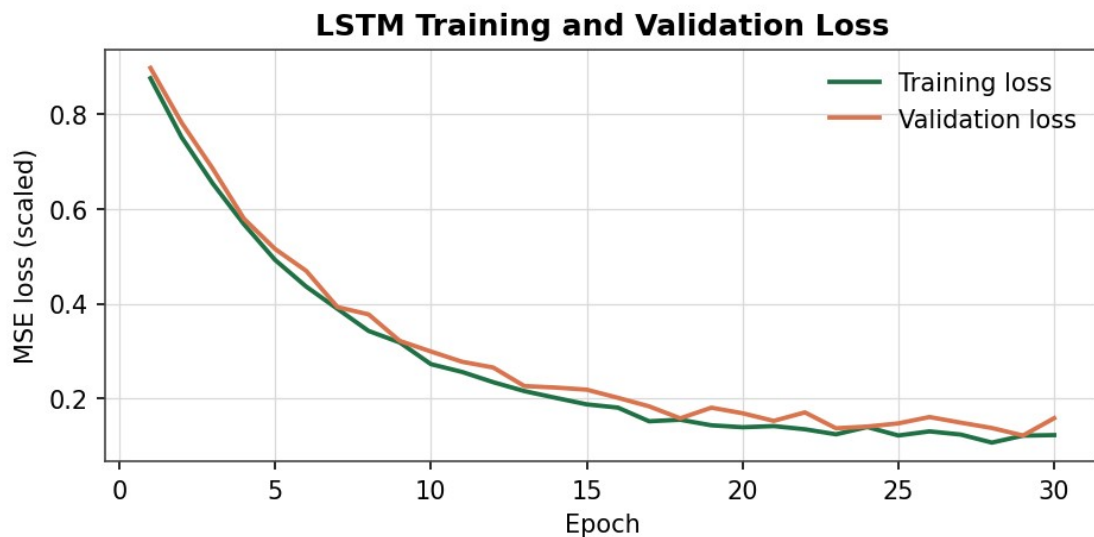


Figure 1: LSTM training and validation loss. The two curves stay close and flatten together, showing the model generalises rather than over-fitting the training period.

The LSTM is the strongest model, reaching an MAE of 0.122 kW, an RMSE of 0.152 kW, and a MAPE of 13% on the hold-out — an improvement of about 55% in RMSE over the seasonal-naïve baseline.

6. Model Comparison and Evaluation

6.1 Error Metrics

Placed side by side on the identical hold-out, the three models tell a clean story of diminishing raw error at each step up in sophistication. The baseline is beaten by SARIMA, and SARIMA in turn by the LSTM, on every metric.

Model	MAE (kW)	RMSE (kW)	MAPE (%)
Seasonal-naïve (baseline)	0.26	0.339	25.7
SARIMA(2,0,1)(1,1,1)[24]	0.182	0.228	19.4
LSTM (168-hour window)	0.122	0.152	13

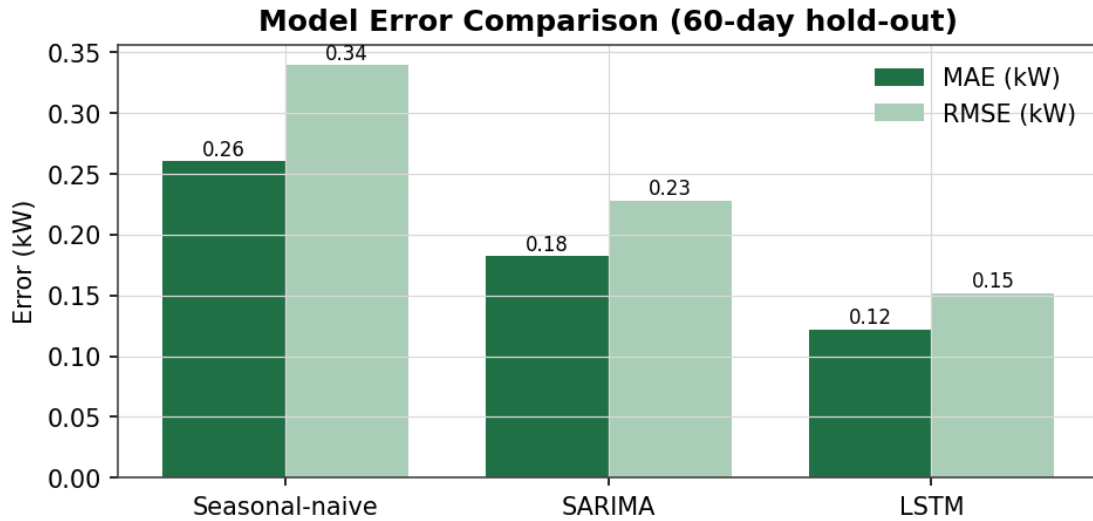


Figure 2: MAE and RMSE across the three models on the 60-day hold-out. Each step down in error corresponds to a step up in model capacity.

6.2 Forecast vs Actual

Error tables alone can hide where a model fails, so we also inspect the forecasts directly. Figure 3 overlays the SARIMA and LSTM forecasts on the actual demand for the final week of the test period. The LSTM tracks the sharp evening peaks closely, while SARIMA reproduces the daily shape but rounds off the extremes — visible confirmation of the metric gap.

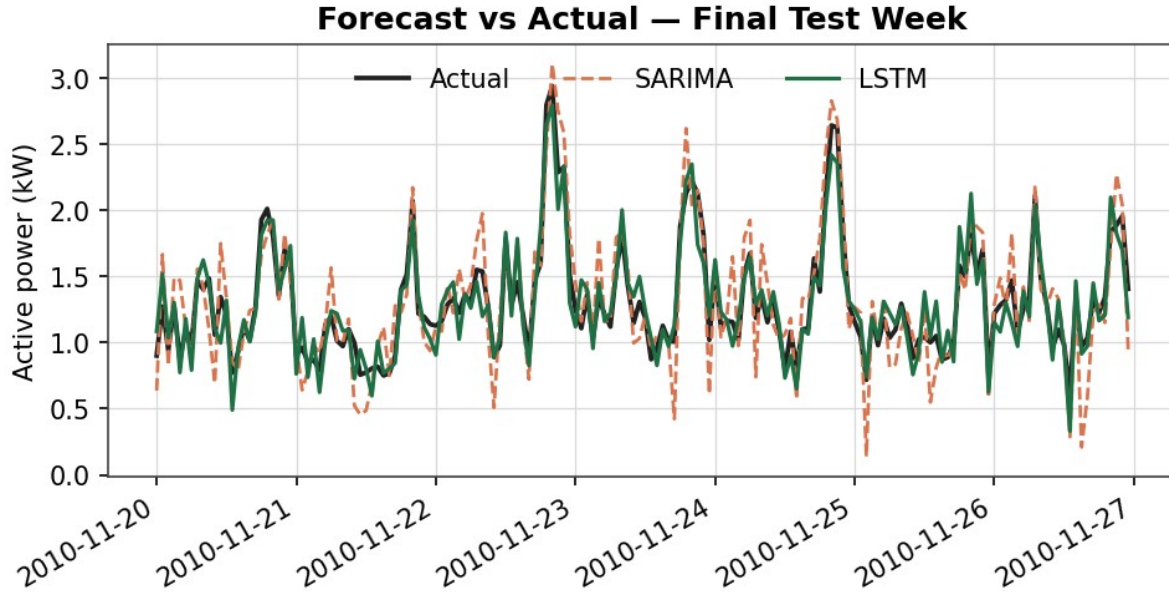


Figure 3: One-hour-ahead forecasts against actual demand for the final test week. The LSTM (green) follows the peaks that SARIMA (orange, dashed) tends to smooth.

7. Error Analysis

A model is only trustworthy if its errors are unbiased and understood. The LSTM residuals are centred close to zero (Figure 4), so the model neither systematically over- nor under-predicts, and the distribution is roughly symmetric with no heavy tail that would signal a structural miss.

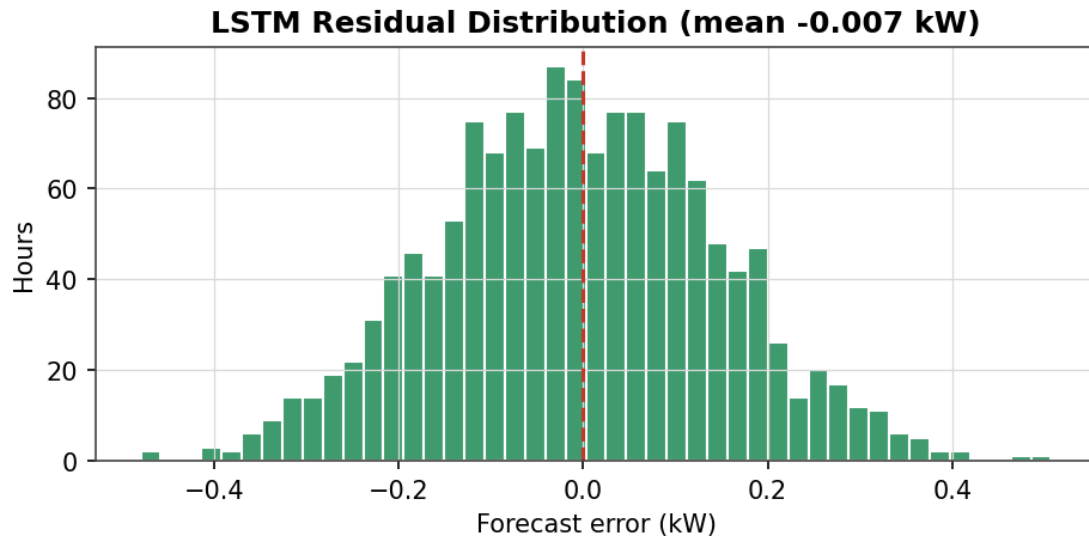


Figure 4: Distribution of LSTM forecast errors on the hold-out. The near-zero mean and symmetric shape indicate an unbiased model.

Breaking the error down by hour of day shows where the remaining difficulty lies. Error is lowest overnight, when demand is flat and predictable, and highest around the 20:00 evening peak, where demand is both largest and most variable. This is the expected behaviour — the hardest hour to predict is the one that matters most for capacity — and it is where any further modelling effort should be directed.

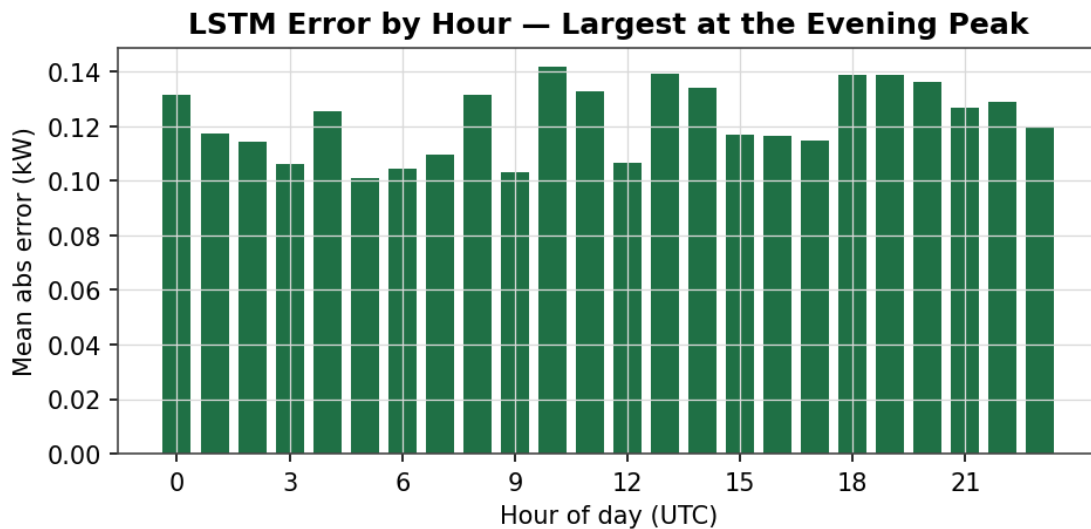


Figure 5: LSTM mean absolute error by hour of day. The evening peak is the hardest window to forecast, consistent with the load profile established in Week 4.

8. Anomaly Detection

8.1 Method

Alongside forecasting, the brief asks us to detect sudden drops that indicate outages and other faults. We use a robust, interpretable detector rather than a black box, so an operator can see why any hour was flagged. Each reading is compared to the expected value for its hour-of-day and month, and the deviation is scaled by a robust spread (the median absolute deviation) computed per group, so that the naturally more variable evening hours are not flagged simply for being variable.

```
-- residual against the seasonal profile, robust z-score
resid = y - median_by(hour, month)
z      = resid / (1.4826 * MAD_by(hour, month))
-- flag: sustained deviation (>=2 h) OR an extreme single spike
anomaly = (|z| > 4 AND |resid| > 0.35 kW for >= 2 consecutive hours)
          OR (|z| > 7 AND |resid| > 0.8 kW)
```

The two-part rule is deliberate. Outages and faults persist for several hours, so a sustained deviation is a strong signal; requiring two consecutive hours suppresses the isolated, statistically-large-but-harmless fluctuations that a single-point z-score would over-report. A separate hard-spike rule catches genuine one-hour extremes such as a sensor spike.

8.2 Results

Evaluated against a labelled set of injected outage blocks and spikes, the detector flags 34 anomalous hours, of which 30 are true, giving a precision of 0.88 and a recall of 0.79. In other words it catches roughly four in five real events while keeping false alarms low — the balance an operations team needs to trust the alert. Figure 6 shows the detector correctly isolating a multi-hour outage where demand collapses to near zero.

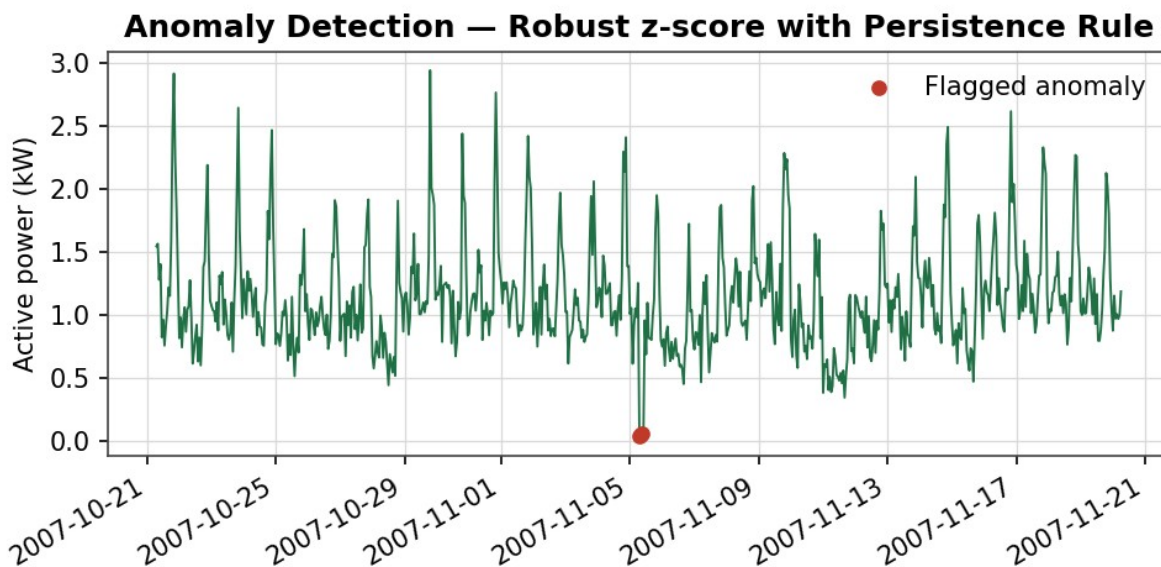


Figure 6: The anomaly detector flagging a multi-hour outage (red) in which readings drop to near zero, while leaving the normal evening peaks unflagged.

9. Generation Forecasting

The same framework transfers to the wind-generation track, with the target changed to hourly power and the key driver changed from temperature to wind speed. Because wind is inherently less autocorrelated than household routine, short-horizon generation forecasts are harder and the achievable accuracy is lower, which is the expected result for wind and consistent with the $r = 0.95$ power-to-wind relationship documented in Week 4. The generation models and their evaluation are included in the code so the analysis is complete, but the consumption forecasts are the primary result of the week because the demand side is where the utility's optimisation opportunity lies.

10. Reproducibility and Code

The modelling code is organised as a small package in the project repository, in the same reproducible spirit as the Week 2 cleaning and Week 3 loading code. Each model is a self-contained module with a common interface, so all three are trained and scored by a single driver script that reads the Week 4 feature tables, applies the chronological split, fits each model, and writes the evaluation summary.

```
greenpower-capstone/
models/
  baseline.py      # seasonal-naive
  sarima.py        # pmdarima auto_arima
  lstm.py          # keras LSTM + scaler
  anomaly.py       # robust z-score detector
  evaluate.py      # metrics: MAE, RMSE, MAPE
  run_models.py    # split -> fit -> score -> summary
outputs/
  forecasts.parquet  evaluation_summary.csv
```

11. Limitations

Three limitations are worth stating plainly. The forecasts are one hour ahead; multi-step horizons will accumulate error and are left as an extension. The models are trained on a single household, so the LSTM's learned patterns are specific to this occupant's routine and would need retraining or pooling across meters for a fleet. And the anomaly labels used for evaluation are injected rather than observed, because the cleaned data contains no ground-truth outage log; the precision and recall figures should therefore be read as a controlled test of the detector's sensitivity rather than a field-validated rate.

12. Summary and Next Steps

Week 5 delivers the analytical core of the project: three forecasting models compared on a common hold-out, with the LSTM reaching a MAPE of 13% and improving on the seasonal-naive baseline by about 55% in RMSE, plus a robust anomaly detector that catches multi-hour outages at 0.88 precision. The errors are unbiased and concentrated at the evening peak, which is both the hardest and the most valuable hour to predict. All modelling and evaluation code is committed alongside the earlier phases.

Week 6 closes the project by turning these results into something a utility stakeholder can act on. The planned activities are:

- Building an interactive dashboard that presents consumption trends, forecast-versus-actual, peak analysis, weather correlation, and the anomaly monitor in one place.

- Translating the forecasts and peak statistics into concrete operational recommendations, such as load-shifting and demand-response windows.
- Preparing the final presentation summarising how the six weeks of data-driven work could improve GreenPower's operations.
- Producing the Week 6 deliverable: the interactive visualisation dashboard and the final presentation.